

Qbase End-to-End Encryption Sync Framework

This document outlines the architecture, cryptographic systems, database permissions, and auto-alignment mechanisms that enable secure, real-time End-to-End Encrypted (E2EE) messaging in Qbase.

1. Architectural Foundations

Qbase implements a robust E2EE system designed on top of **Google Tink** and **Appwrite**. The framework guarantees that no intermediary server (including Appwrite) can read message payloads. Only the intended recipients possess the mathematical keys to decrypt them.

- **Google Tink:** Used to manage key generation, hybrid asymmetric encryption/decryption, and symmetric payload encryption.
 - **Appwrite Core:** Used for real-time synchronization, document storage, and Document Level Security (DLS).
-

2. Key Generation and Storage

The E2EE framework utilizes a hybrid cryptosystem combining ECIES asymmetric encryption and AES symmetric encryption.

2.1 Local Keyset Management

When a user launches the app or registers, the device checks for a local Tink keyset in private storage.

- **Storage Location:** The keyset is persisted inside SharedPreferences using the Android Keyset Manager.
- **Hardware Security:** The keyset is encrypted using an Android Keystore master key (using ``android-keystore://`` scheme).
- **Asymmetric Template:** The framework uses **ECIES P256 HKDF HMAC SHA256 AES128 GCM** for key exchange.

2.2 Public Profile Registration

To allow other users to encrypt messages for them, the client registers their public key Base64 string on the Appwrite remote database.

- **Collection:** ``users`` (Public Profiles)
 - **Attribute:** ``publicKey`` (Base64 string representing Tink's public keyset handle)
-

3. The Secure Permission Hierarchy

To protect metadata and user profiles while allowing E2EE keys to be shared, Qbase enforces a strict permission model using Appwrite Document Level Security (DLS).

3.1 Profile Permissions (users)

- **Read:** ``Role.users()`` (any authenticated user can fetch a peer's public key by Friend Code).
- **Write/Update:** ``Role.user(userId)`` (only the profile owner can modify their profile metadata or public key).

3.2 Private settings (user_private_settings)

- **Read/Write:** ``Role.user(userId)`` (strictly private to the user; contains E2EE encrypted backups).

3.3 Message Permissions (messages)

- **Read/Write:** ``Role.users()`` with real-time observer filtering based on participant IDs.

4. Encrypted Messaging Lifecycle

When Alice sends an encrypted message to Bob, the following lifecycle is executed:

4.1 Key Exchange & Session Key Generation

1. Alice's device generates a random **AES-128-GCM session key** specifically for the message payload.
2. Alice fetches Bob's public key from the remote ``users`` collection in Appwrite.
3. Alice's device encrypts the session key using **Bob's public key** and Alice's own public key.
4. Alice generates a JSON map called ``wrappedKeys`` representing:
 - ``aliceId`` -> ``wrappedSessionKeyForAlice``
 - ``bobId`` -> ``wrappedSessionKeyForBob``

4.2 Payload Encryption & Upload

1. Alice encrypts the message payload (text or file meta) using the generated AES session key.
2. Alice uploads the encrypted payload (ciphertext) and ``wrappedKeys`` JSON map as a single document to the ``messages`` collection.

4.3 Decryption at Receiver

1. Bob's device receives the real-time message event via Appwrite Realtime stream.
2. Bob's device parses the ``wrappedKeys`` map and extracts his own wrapped key ``wrappedKeys[bobId]``.
3. Bob's device decrypts the wrapped key using his **local private keyset handle** (unwrapping the session key).
4. Bob's device decrypts the ciphertext payload using the unwrapped session key and displays the plaintext to Bob.

5. Self-Healing E2EE Auto-Alignment

A primary E2EE failure mode in multi-device or reinstalled environments is **Key Mismatch**. If Bob reinstalls

the app, his local SharedPreferences are cleared, and a new keyset is generated. If Appwrite still contains Bob's old public key, Alice will encrypt messages using the old key, causing Bob to see a decryption error.

To solve this, Qbase implements a **Self-Healing Key Alignment pipeline**:

5.1 Background Verification

Whenever a user opens the chat screen or initiates synchronization, a background worker is launched.

1. The client fetches their own active user profile from Appwrite `/users/userId`.
2. The client compares the remote public key with the active local E2EE public key.
3. If there is a mismatch (or the remote key is missing), the client **automatically updates Appwrite** with their current local public key.

5.2 Real-time Recipient Alignment

When Alice prepares to send a message to Bob:

1. The client always fetches the latest public key from Appwrite.
2. If the retrieved remote key differs from Alice's local SQLite cache, Alice's device **automatically overwrites its cache** and encrypts the session key with the correct active public key.

This double-sided self-healing architecture guarantees that E2EE keys are always synchronized dynamically across all active clients in real-time.